NATIONAL UNIVERSITY of Computer & Emerging Sciences

CS1004 - Object Oriented Programming

LAB # 1

REVISING PROGRAMMING FUNDAMENTALS

Lab Objectives :

- Review fundamental programming concepts
- Practice function implementation and usage
- Strengthen problem-solving skills with control structures
- Work with arrays and basic algorithms

LAB TASKS

Task 1: Leap Year Checker

Write a function isLeapYear() that takes a year as parameter and returns 1 if it's a leap year, otherwise 0. A year is leap if:

- Divisible by 4 AND not divisible by 100, OR
- Divisible by 400

Test with 5 years in main.
Sample Input:
Enter 5 years: 2000 2100 2024 2023 1900
Sample Output:
2000 is a Leap Year
2100 is not a Leap Year
2024 is a Leap Year
2023 is not a Leap Year
1900 is not a Leap Year

Task 2: Convert Seconds

Create a function convertSeconds() that takes total seconds as parameter and displays the time in hours, minutes, and seconds format.
Sample Input:
Enter seconds: 3725
Sample Output:
3725 seconds = 1 hour(s), 2 minute(s), 5 second(s)

Task 3: Array Minimum Finder

Write a function findMin() that takes an array and its size as parameters and returns the minimum element along with its index (you can use a structure or pass index by reference).
Sample Input:
Enter 8 numbers: 45 23 67 12 89 34 56 8
Sample Output:
Minimum element: 8
Found at index: 7

Task 4: Number Pyramid

Create a function printNumberPyramid() that takes an integer n and prints a number pyramid of n rows.
Sample Input:
Enter number of rows: 5
Sample Output:
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5

Task 5: Sum of Series

Write a function sumSeries() that calculates and returns the sum of series: 1 + 2 + 3 + ... + n.
Sample Input:
Enter n: 10
Sample Output:
Sum of series 1+2+3+...+10 = 55

Task 6: Strong Number Checker

Write a function isStrong() that checks if a number is a strong number. A strong number is a number whose sum of factorial of digits equals the original number (e.g., 145 = 1! + 4! + 5!). Find all strong numbers between 1 and 200.

Sample Output:

Strong numbers between 1 and 200:
1
2
145

Task 7: Selection Sort and Analysis

Implement the following functions:

- selectionSort(): Sorts array using selection sort algorithm
- countComparisons(): Counts number of comparisons made during sorting (modify selectionSort to track this)
- displayArray(): Displays array

Sort an array of 12 numbers and display comparison count.

Sample Input:

Enter 12 numbers: 64 25 12 22 11 90 88 45 50 3 78 33

Sample Output:

Original Array: 64 25 12 22 11 90 88 45 50 3 78 33
Sorted Array: 3 11 12 22 25 33 45 50 64 78 88 90
Total Comparisons: 66

Task 8: Anagram Checker

Create the following functions:

- toLowerCase(): Converts string to lowercase
- sortString(): Sorts characters in a string alphabetically
- areAnagrams(): Checks if two strings are anagrams (contain same characters in different order)

Test with multiple string pairs.

Sample Input:

Enter first string: Listen
Enter second string: Silent

Sample Output:

"Listen" and "Silent" are Anagrams
Enter first string: Hello
Enter second string: World
"Hello" and "World" are not Anagrams

Task 9: Matrix Operations Suite

Create a comprehensive matrix operations program with these functions:

- inputMatrix(): Input matrix elements
- displayMatrix(): Display matrix
- addMatrices(): Add two matrices
- multiplyMatrices(): Multiply two matrices (ensure dimensions are compatible)
- findDeterminant(): Find determinant of 2x2 matrix
- isDiagonal(): Check if matrix is diagonal
- isIdentity(): Check if matrix is identity matrix

Perform operations on 3×3 matrices.
Sample Input:
Matrix A (3x3):
1 2 3
4 5 6
7 8 9
Matrix B (3x3):
9 8 7
6 5 4
3 2 1
Sample Output:
Matrix A:
1 2 3
4 5 6
7 8 9
Matrix B:
9 8 7
6 5 4
3 2 1
Sum (A + B):
10 10 10
10 10 10
10 10 10
Product (A × B):
30 24 18
84 69 54
138 114 90
Matrix A is not Diagonal
Matrix A is not Identity

Task 10: Student Grade Management System

Design a comprehensive grade management system with these functions:

- inputStudentData(): Enter data for n students (name, roll no, marks in 5 subjects)

- calculateTotalAndAverage(): Calculate total and average for each student
- assignGrade(): Assign grade based on average (A≥90, B≥80, C≥70, D≥60, F<60)
- findTopper(): Find student with highest total marks
- countFailures(): Count students who failed (any subject <40)
- displaySubjectAverage(): Calculate and display average of each subject
- displayPassPercentage(): Calculate and display pass percentage of class
- sortByTotal(): Sort students by total marks in descending order
- displayAll(): Display complete report

Manage data for 5 students with 5 subjects each.

Sample Input:

Enter data for 5 students:
Student 1:
Name: Ali
Roll No: 101
Marks (5 subjects): 85 90 78 88 92
Student 2:
Name: Sara
Roll No: 102
Marks (5 subjects): 92 88 95 90 87
Student 3:
Name: Ahmed
Roll No: 103
Marks (5 subjects): 65 70 68 72 75
Student 4:
Name: Fatima
Roll No: 104
Marks (5 subjects): 78 82 85 80 88
Student 5:
Name: Hassan
Roll No: 105
Marks (5 subjects): 35 45 50 38 42

Sample Output:

STUDENT GRADE REPORT
==========

Roll No	Name	Sub1	Sub2	Sub3	Sub4	Sub5	Total	Average	Grade
102	Sara	92	88	95	90	87	452	90.40	A
101	Ali	85	90	78	88	92	433	86.60	B
104	Fatima	78	82	85	80	88	413	82.60	B
103	Ahmed	65	70	68	72	75	350	70.00	C
105	Hassan	35	45	50	38	42	210	42.00	F

CLASS STATISTICS
==========
Class Topper: Sara (Roll No: 102) – Total: 452

Subject-wise Average:
Subject 1: 71.00
Subject 2: 75.00
Subject 3: 75.20
Subject 4: 73.60
Subject 5: 76.80
Students with failures: 1
Pass Percentage: 80.00%